

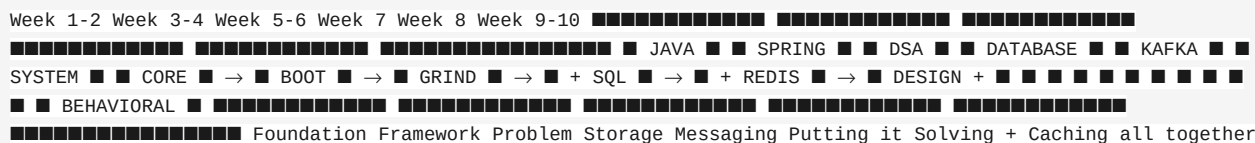
Backend Developer Interview Preparation Roadmap

A no-fluff, week-by-week plan going from Java fundamentals to system design.

Estimated time: 8-10 weeks (2-3 hours/day).

Adjust pace based on your comfort — skip what you already know, double down on weak areas.

The Big Picture



Phase 1: Java Core Revision (Week 1-2)

Goal: Be able to explain any Java concept as if teaching a junior developer.

Week 1 — Language Fundamentals & OOP

Day 1-2: OOP Done Right

- The 4 pillars — but with depth:
- **Encapsulation:** Not just "private fields + getters." Understand *why* — protecting invariants. If a `BankAccount` class lets you set balance to -500 directly, encapsulation is broken even with getters/setters.
- **Inheritance vs Composition:** When to use which. "Favor composition over inheritance" — know why (fragile base class problem).
- **Polymorphism:** Runtime (method overriding) vs Compile-time (method overloading). How does the JVM resolve which method to call?
- **Abstraction:** Abstract classes vs Interfaces. When Java 8 added default methods, the line blurred — know the current rules.
- SOLID principles — one real example each, not textbook definitions.

Day 3-4: Generics, Collections & Internals

- Generics: `<T>`, `<? extends T>`, `<? super T>` — the PECS rule (Producer Extends, Consumer Super)
- **HashMap deep dive:** How `put()` works, hashing, collisions, treeify at 8 nodes, resize at 0.75 load factor
- **ArrayList vs LinkedList:** Not just "one is array, one is linked list." When is `LinkedList` actually better? (Almost never — cache locality matters)
- **HashSet:** It's literally a `HashMap` with dummy values. Know this.
- **TreeMap:** Red-Black tree underneath. Sorted keys. $O(\log n)$ operations.

Day 5-7: Strings, Immutability & Memory

- String Pool — why "hello" == "hello" is true but new String("hello") == new String("hello") is false
- String vs StringBuilder vs StringBuffer — and why this still gets asked
- How equals() and hashCode() contract works — what breaks if you override one but not the other
- Pass-by-value in Java — Java is ALWAYS pass-by-value (the "value" for objects is the reference)

Week 2 — Concurrency, JVM & Java 8+

Day 1-2: Multithreading & Concurrency

- Thread lifecycle: NEW → RUNNABLE → RUNNING → BLOCKED/WAITING → TERMINATED
- `synchronized` keyword — method-level vs block-level, what's the monitor object?
- `volatile` — visibility guarantee, but no atomicity (can't replace synchronization for `count++`)
- **ConcurrentHashMap**: How it avoids locking the whole map (CAS for empty buckets, synchronized per-node)
- **Fail-fast vs Fail-safe iterators** — and which collections use which
- `ExecutorService`, `ThreadPoolExecutor` — core pool size, max pool size, work queue. What happens when the queue is full?
- `CompletableFuture` — chaining async tasks, `thenApply` vs `thenCompose`, exception handling

Day 3-4: JVM Internals

- Memory areas: Heap (Young Gen → Eden + Survivor, Old Gen), Metaspace (replaced PermGen in Java 8), Stack (per thread)
- Garbage Collection: Mark → Sweep → Compact
- **G1 GC**: Region-based, collects garbage-first regions, predictable pause times
- Class loading: Bootstrap → Extension → Application class loaders. Parent delegation model.
- How to identify a memory leak: heap dumps, `jmap`, Eclipse MAT, GC logs

Day 5-6: Java 8 to 21 Features

- **Java 8**: Lambdas, Streams (`map/filter/reduce/collect`), Optional, `@FunctionalInterface`
- **Java 11**: `var` keyword, `HttpClient`, `String` new methods (`isBlank`, `strip`, `lines`)
- **Java 17**: Sealed classes, Pattern matching for `instanceof`, Records, Text blocks
- **Java 21**: Virtual Threads (Project Loom), Sequenced Collections, Pattern matching for switch
- Focus on **Records** and **Virtual Threads** — these are hot interview topics in 2025-26

Day 7: Practice Day

- Take 20 common "Java tricky questions" and answer them without looking anything up
- Write a thread-safe singleton (enum-based, double-checked locking)
- Implement a simple producer-consumer using `BlockingQueue`

Phase 2: Spring Boot Mastery (Week 3-4)

Goal: Understand Spring Boot end-to-end, not just how to use it but *how it works internally*.

Week 3 — Core Spring & Spring Boot

Day 1-2: Spring Core Concepts

- **IoC (Inversion of Control)**: You don't create objects. Spring creates and injects them.
- **Dependency Injection**: Constructor injection (preferred) vs Field injection vs Setter injection — and why constructor injection is best (immutability, testability, no hidden dependencies)
- **Bean lifecycle**: Constructor → `@PostConstruct` → ready → `@PreDestroy` → garbage collected
- **Bean scopes**: Singleton (default), Prototype, Request, Session — when to use each
- `@Component` vs `@Service` vs `@Repository` vs `@Controller` — all are `@Component`, the difference is semantic + `@Repository` adds exception translation

Day 3-4: Spring Boot Auto-Configuration & Internals

- How `@SpringBootApplication` works: it's `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`
- Auto-configuration: Spring Boot reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` and conditionally creates beans based on what's on the classpath
- `@Conditional` annotations: `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty`
- **Profiles**: `application.yml` vs `application-dev.yml` vs `application-prod.yml`. How `@Profile` annotation works.
- **Actuator**: Health checks, metrics, environment info — and how to create custom endpoints

Day 5-7: Spring Boot Web & Data

- REST API design: proper HTTP methods, status codes, request/response DTOs
- Exception handling: `@ControllerAdvice` + `@ExceptionHandler` for global error handling
- Validation: `@Valid`, `@NotNull`, `@Size`, custom validators
- **Spring Data JPA:**
- How `@Transactional` works (AOP Proxy — know the self-invocation trap)
- N+1 query problem — what it is, how to detect, how to fix (`@EntityGraph`, `JOIN FETCH`)
- Lazy vs Eager loading — and why `LazyInitializationException` happens
- Optimistic Locking with `@Version`

Week 4 — Security, Testing & Advanced Topics

Day 1-2: Spring Security

- Authentication vs Authorization
- Filter chain concept — how requests pass through security filters
- JWT-based authentication flow:
 1. User logs in → server returns JWT
 2. User sends JWT in `Authorization: Bearer <token>` header
 3. Server validates JWT, extracts user, sets `SecurityContext`
- `@PreAuthorize`, `@Secured` — method-level security
- CORS configuration

Day 3-4: Testing

- Unit testing: `@MockBean`, `@InjectMocks`, Mockito basics
- Integration testing: `@SpringBootTest`, `@DataJpaTest`, `@WebMvcTest`
- Test slices — why you don't need to load the full context for every test
- MockMvc for testing controllers without starting a server

Day 5-7: Microservices Patterns (Theory + Spring Cloud)

- Service discovery (Eureka / Consul)
- API Gateway pattern (Spring Cloud Gateway)
- Circuit Breaker (Resilience4j) — the states: CLOSED → OPEN → HALF_OPEN
- Config Server (externalized configuration)
- Distributed tracing (Micrometer + Zipkin)
- Inter-service communication: REST (sync) vs Messaging (async)

Phase 3: DSA Grind (Week 5-6)

Goal: Solve 50-60 problems across key patterns. Quality over quantity.

The Pattern-Based Approach

Don't solve random problems. Group them by pattern so your brain builds templates.

Week 5 — Arrays, Strings, Hashing, Sliding Window

Day 1: Two Pointers (5 problems)

- Two Sum (sorted array)
- Container With Most Water
- 3Sum

- Remove Duplicates from Sorted Array
- Trapping Rain Water

Day 2-3: Sliding Window (6 problems)

- Maximum Sum Subarray of Size K
- Longest Substring Without Repeating Characters
- Max Consecutive Ones III (flip k zeros)
- Minimum Window Substring
- Fruit Into Baskets
- Permutation in String

Day 4: Hashing (4 problems)

- Two Sum (unsorted)
- Group Anagrams
- Longest Consecutive Sequence
- Subarray Sum Equals K

Day 5-6: Binary Search (5 problems)

- Search in Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Search a 2D Matrix
- Koko Eating Bananas
- Median of Two Sorted Arrays

Day 7: Stacks & Queues (4 problems)

- Valid Parentheses
- Min Stack
- Next Greater Element
- Implement Queue using Stacks

Week 6 — Trees, Graphs, DP, Linked Lists

Day 1-2: Trees (6 problems)

- Level Order / Zig-Zag Traversal
- Validate BST
- Lowest Common Ancestor
- Diameter of Binary Tree
- Serialize and Deserialize Binary Tree
- Binary Tree Maximum Path Sum

Day 3: Linked Lists (4 problems)

- Reverse Linked List
- Reverse in Groups of K
- Detect and Remove Cycle
- Merge K Sorted Lists

Day 4-5: Graphs (5 problems)

- BFS / DFS traversal
- Number of Islands
- Course Schedule (Topological Sort)
- Shortest Path (Dijkstra's)
- Clone Graph

Day 6-7: Dynamic Programming (6 problems — focus on these patterns)

- Fibonacci pattern: Climbing Stairs, House Robber
- 0/1 Knapsack: Partition Equal Subset Sum, Target Sum
- Longest Common Subsequence
- Coin Change
- Longest Increasing Subsequence
- Edit Distance

DSA Tips

- Solve on paper first, then code — interviewers want to see your thought process
- Always state brute force first, then optimize
- Practice explaining your approach out loud as you code
- Time yourself: 20 min for medium, 30 min for hard

Phase 4: Databases & SQL (Week 7)

Goal: Go from "I can write queries" to "I understand how the DB engine works."

Day 1: SQL Mastery

- JOINS: INNER, LEFT, RIGHT, FULL, CROSS, SELF — draw Venn diagrams, know when each is used
- GROUP BY + HAVING vs WHERE — execution order matters
- Window functions: ROW_NUMBER(), RANK(), DENSE_RANK(), LAG(), LEAD(), SUM() OVER()
- Subqueries vs CTEs — when to use which, performance implications
- Practice: solve 10 medium SQL problems on LeetCode or HackerRank

Day 2: Indexing Deep Dive

- Clustered vs Non-Clustered indexes
- B+ Tree structure — why databases use B+ Trees (not binary trees or hash maps)
- Composite indexes — leftmost prefix rule
- Covering indexes — how to eliminate bookmark lookups
- Why a query can be slow even with an index (6 reasons — functions on columns, low selectivity, stale stats, fragmentation, type mismatch, SELECT *)

Day 3: Transactions & Concurrency

- ACID properties — know each with examples
- Isolation levels: READ_UNCOMMITTED → READ_COMMITTED → REPEATABLE_READ → SERIALIZABLE
- What problems each level prevents: dirty reads, non-repeatable reads, phantom reads
- Optimistic vs Pessimistic locking — when to use which
- Deadlocks: how they happen, how to prevent, how to detect

Day 4: Database Design

- Normalization: 1NF → 2NF → 3NF → BCNF — with examples
- When to denormalize (read-heavy systems, reporting tables)
- SQL vs NoSQL decision framework:
- **SQL:** Structured data, complex queries, ACID needed, relationships matter
- **NoSQL:** Flexible schema, horizontal scaling, high write throughput, denormalized reads

Day 5: Query Optimization

- How to read an EXPLAIN plan (MySQL/PostgreSQL/Oracle)
- Common anti-patterns: N+1 queries, SELECT *, missing indexes, OR in WHERE
- Connection pooling: HikariCP — why and how to tune it
- Pagination: Offset-based (slow for deep pages) vs Cursor-based (consistent performance)

Day 6-7: Practice

- Design a schema for an e-commerce system (users, products, orders, payments)
- Write queries: "Top 5 customers by spending in last 30 days", "Products never ordered", "Running total of daily revenue"
- Optimize a slow query given an EXPLAIN plan

Phase 5: Kafka + Redis (Week 8)

Goal: Understand these not as buzzwords, but as tools you can reason about in system design.

Kafka (Day 1-3)

Day 1: Kafka Fundamentals

- What Kafka is: a distributed, append-only commit log
- Core concepts: Topics, Partitions, Brokers, Producers, Consumers
- Why partitions matter: parallelism + ordering guarantee within a partition
- How messages are stored: on disk, sequential writes (that's why it's fast)
- Retention: time-based or size-based, messages are NOT deleted after consumption
- Consumer offsets: Kafka tracks what each consumer group has read

Day 2: Consumer Groups & Delivery Semantics

- Consumer Groups: how partitions are assigned to consumers, max parallelism = partition count
- Rebalancing: when it happens, eager vs cooperative, how to minimize impact
- Delivery guarantees:
 - At-most-once: read → commit offset → process (might lose messages)
 - At-least-once: read → process → commit offset (might duplicate)
 - Exactly-once: idempotent producer + transactional writes + offset commit in transaction

Day 3: Kafka in System Design

- Event-Driven Architecture: how services communicate via events instead of API calls
- Event Sourcing: storing state changes as a sequence of events
- CQRS: Command Query Responsibility Segregation — write to one model, read from another
- Common patterns:
 - Order placed → Payment service, Inventory service, Notification service all consume
 - Dead Letter Queue (DLQ): where failed messages go for manual inspection
 - Saga pattern: coordinating distributed transactions via events

Redis (Day 4-6)

Day 4: Redis Fundamentals

- What Redis is: in-memory key-value data structure store
- Data types and when to use each:
 - **String:** Caching a serialized object, counters, session tokens
 - **Hash:** Storing object fields (like a mini-table row) — HSET user:123 name "Satjit" age 26
 - **List:** Message queues, recent activity feeds — LPUSH / RPOP
 - **Set:** Tags, unique visitors, mutual friends — SINTER, SUNION

- **Sorted Set (ZSet):** Leaderboards, rate limiters, priority queues — ZADD, ZRANK
- TTL (Time To Live): auto-expiry of keys — fundamental for caching

Day 5: Redis Patterns

- **Caching strategies:**
- **Cache-Aside (Lazy):** App checks cache → miss → query DB → store in cache → return. Most common.
- **Write-Through:** Write to cache AND DB simultaneously. Consistent but slower writes.
- **Write-Behind:** Write to cache, async write to DB. Fast writes but risk of data loss.
- **Cache stampede:** 1000 requests hit at the exact moment cache expires → all hit DB. Fix: mutex lock, early refresh, staggered TTL.
- **Distributed Locking:** SET lockKey uniqueId NX EX 5 — used for the double-booking problem, rate limiters
- **Rate Limiting with Sorted Set:**
`ZADD rate:user123 <timestamp> <requestId> ZREMRANGEBYSCORE rate:user123 0 <timestamp - window> ZCARD rate:user123 → if > limit, reject`
- **Session Storage:** Store session data in Redis instead of server memory → enables stateless app servers behind a load balancer
- **Pub/Sub:** Real-time notifications, chat messages (but no persistence — use Kafka for durability)

Day 6: Redis in Production

- **Persistence:** RDB snapshots (periodic) vs AOF (every write logged). Trade-off: performance vs durability.
- **Replication:** Master-Replica setup for read scaling and failover
- **Redis Cluster:** Sharding across multiple nodes, 16384 hash slots
- **Eviction policies:** When memory is full, what gets deleted?
- `allkeys-lru` — Least Recently Used across all keys (most common for caching)
- `volatile-lru` — LRU only among keys with TTL
- `noeviction` — Return errors when full (use for critical data)
- **Common pitfalls:**
- Big keys: a single hash with 10M fields — blocks Redis during operations
- Hot keys: one key getting 90% of traffic — use read replicas or client-side caching
- Missing TTL: cache grows forever → OOM

Day 7: Integrate It All

- Design a simple system using all three:

User places order on API (Spring Boot) ■ ■■■ Check Redis cache for product availability ■■■ Write order to MySQL (with @Transactional) ■■■ Publish "OrderCreated" event to Kafka ■ ■■■ Payment Service (Kafka consumer) → process payment ■■■ Inventory Service (Kafka consumer) → reduce stock, invalidate Redis cache ■■■ Notification Service (Kafka consumer) → send SMS

Phase 6: System Design + Behavioral (Week 9-10)

Goal: Tie everything together. Show you can design systems, not just code features.

Week 9 — Low-Level Design (LLD)

Practice these designs (pick 4-5):

- Parking Lot System
- URL Shortener (like bit.ly)
- Rate Limiter
- Movie Ticket Booking (BookMyShow)
- Notification Service
- Splitwise (expense sharing)
- Elevator System

For each, practice:

1. Clarify requirements (functional + non-functional)
2. Identify core entities and relationships
3. Define interfaces / APIs
4. Write the key classes with proper design patterns
5. Handle concurrency / edge cases

Design Patterns to know:

- Singleton, Factory, Builder, Strategy, Observer, Decorator, Adapter
- Don't memorize definitions — know when you'd USE each one

Week 10 — High-Level Design (HLD) + Behavioral

Practice these designs (pick 3-4):

- Design a Payment System (very relevant for Paytm)
- Design a URL Shortener at scale
- Design a Chat System (WhatsApp)
- Design a Notification System
- Design an E-Commerce Order Flow

For each, follow this framework:

1. **Requirements:** Functional (what it does) + Non-functional (scale, latency, availability)
2. **Estimation:** QPS, storage, bandwidth — back-of-envelope math
3. **High-Level Architecture:** Draw the boxes (API Gateway → Services → DB/Cache/Queue)
4. **Deep Dive:** Pick 2-3 components and go deep (data model, API design, caching strategy)
5. **Trade-offs:** "I chose X over Y because..."

Behavioral (ongoing — prepare 4-5 STAR stories):

- A time you optimized something (query, service, process)
- A time you dealt with a production incident
- A time you disagreed with a teammate/lead
- A time you learned something new quickly
- A time you mentored someone or took ownership

Daily Routine Template

[illegible]

Resource Recommendations

Topic	Best Resource	Why
-------	---------------	-----

Java Internals	Baeldung.com	Concise, code-heavy, always up to date
Spring Boot	Official Guides + Marco Behler's blog	Practical, not abstract
DSA	NeetCode.io (Roadmap)	Pattern-based, curated 150 problems
SQL	LeetCode SQL 50	Real interview questions
System Design	Alex Xu's "System Design Interview" book	Clear diagrams, practical scale numbers
Kafka	Confluent's official docs + tutorials	Written by Kafka's creators
Redis	Redis University (free)	Hands-on labs included
Mock Interviews	Pramp.com (free)	Peer-to-peer, real-time practice

The Priority Matrix

If you're short on time and can only do 4 weeks, focus on these in order:

MUST KNOW (asked in 90% of interviews): ✓ Java Core (HashMap, Concurrency, Java 8+ features) ✓ Spring Boot (@Transactional, Security basics, REST) ✓ DSA (Sliding Window, Trees, Binary Search – 30 problems) ✓ SQL (JOINS, indexes, transactions) STRONG ADVANTAGE (asked in 60% of SSE interviews): ✓ Kafka (consumer groups, exactly-once, basic architecture) ✓ Redis (caching strategies, data types, distributed locking) ✓ LLD (at least 2 designs practiced) DIFFERENTIATOR (separates Senior from Mid-level): ✓ HLD (payment system, notification system) ✓ JVM internals (GC, memory model) ✓ Microservices patterns (circuit breaker, saga, CQRS)

You've got the foundation from Amdocs — the catalog systems, Oracle DB work, and Java experience. This roadmap fills the gaps that product companies look for. Stick to the plan, and you'll walk into any backend SSE interview with confidence.